

Apostila de Programação em Arduino com C++ Iniciante

Silas Reuel da Silva

Nome:

Bauru - 2025

Introdução.....	3
Configurando o Ambiente de Desenvolvimento para Arduino.....	4
Instalação da IDE.....	4
Configuração da placa na IDE.....	4
Testando a instalação.....	4
Sem a placa Arduino.....	4
Fundamentos da Programação no Arduino.....	6
Convenções para nomeação.....	6
Estrutura de um código - setup() e loop().....	6
Variáveis e tipos de dados.....	7
Números inteiros.....	7
Números com ponto flutuante - decimais.....	7
Tipos lógicos (Booleanos).....	7
Strings e Arrays.....	8
Arrays.....	8
Arrays Bidimensionais.....	9
Operadores Básicos.....	10
Adição (+).....	10
Subtração (-).....	10
Divisão (/).....	10
Multiplicação (*).....	11
Controle de Fluxo.....	12
Operadores lógicos.....	12
Operadores relacionais.....	12
Estruturas condicionais.....	13
If e else.....	13
Switch.....	14
Laços de Repetição.....	16
For.....	16
For Aninhado.....	16
While.....	17
Do While.....	19
Funções.....	20
Eletrônica Básica para Arduino.....	22
Prática.....	23
Potenciação.....	23
Blink LED.....	24
Buzzer.....	25
Servo Motor.....	27

Introdução

O Arduino revolucionou o mundo da eletrônica e da programação, tornando acessível a criação de projetos interativos para iniciantes e profissionais. Combinando hardware e software de código aberto, essa plataforma permite o desenvolvimento de protótipos e soluções inovadoras para diversas áreas, como automação, robótica e IoT (Internet das Coisas).

Nesta apostila, você aprenderá desde os conceitos básicos do Arduino até a programação e a criação de projetos práticos. Serão abordados temas como configuração do ambiente de desenvolvimento, estrutura do código em C++, uso de sensores e atuadores, além de dicas para solucionar problemas comuns.

Configurando o Ambiente de Desenvolvimento para Arduino

Instalação da IDE

A **Arduino IDE** (Integrated Development Environment) é o software oficial para programar placas Arduino.

Vá até o link oficial: <https://www.arduino.cc/en/software> e baixe-o.

Conectando o Arduino ao Computador

Use um cabo USB adequado para conectar o Arduino ao computador. A maioria das placas usa USB-A para USB-B (Arduino Uno).

O sistema operacional deve detectar automaticamente a placa. Se estiver no Windows, pode ser necessário instalar drivers adicionais (a própria IDE Arduino faz isso na maioria dos casos).

Configuração da placa na IDE

Abra o Arduino IDE

No menu superior, clique em Ferramentas > Placa e selecione o modelo da sua placa (por exemplo, Arduino Uno).

Ainda em Ferramentas, clique em Porta e selecione a porta serial onde a placa foi conectada (exemplo: COM3 no Windows).

💡 *Se a porta não aparecer, tente desconectar e reconectar o cabo ou reiniciar a IDE.*

Testando a instalação

Agora vamos testar se tudo está funcionando com o código mais básico do Arduino: o Blink, que faz um LED piscar.

Na IDE, vá em Arquivo > Exemplos > 01.Basics > Blink.

O código abrirá automaticamente. Clique no botão Carregar (ícone de seta para a direita).

Se tudo estiver correto, o LED da placa começará a piscar.

Sem a placa Arduino

Se você não tem uma placa Arduino, não se preocupe, use o <https://www.tinkercad.com/>. Este site simula o uso do Arduino e programação, com

as opções de programar em Blocos ou em C++. Essa apostila abordará a programação convencional (código em C++) e uso do tinkercad. Não se preocupe, o que for ensinado nessa apostila também vale para o Arduino.

Fundamentos da Programação no Arduino

A linguagem C++ é **case-sensitive**, ou seja, diferencia maiúsculas de minúsculas, a variável 'numero' é diferente de 'Numero'.

Não use caracteres especiais. Apenas se for imprimir algo no terminal.

Também não é necessário o uso de indentação, mas é uma convenção para legibilidade do código.

Convenções para nomeação

Variáveis e funções >> camelCase, a primeira letra da primeira palavra é minúscula e das outras maiúsculas;

Constantes e definições >> UPPER_CASE_SNAKE_CASE, todas as palavras maiúsculas separadas por *underline* (_);

Estruturas e classes >> PascalCase, a primeira letra de cada palavra deve ser maiúscula.

Estrutura de um código - setup() e loop()

A estrutura básica de um código para Arduino contém as funções setup() e loop(). A função Setup é obrigatória e é declarada da seguinte maneira:

```
void setup()
{
    //Código do setup
}
```

Qualquer código que estiver dentro do setup (), ou seja, entre chaves ({ }) é executado uma única vez no início do seu programa.

Essa função é útil para configurar o Arduino, é aqui que você coloca as configurações iniciais, como por exemplo se um LED começa desligado ou ligado, quais são os pinos de entrada e saída, entre outras coisas.

O Loop também é obrigatório em um programa para Arduino. A sua declaração é feita da seguinte forma:

```
void loop()
{
    //Código do loop
}
```

A grande maioria do seu código será executado dentro dessa seção. Após a execução do setup(), o loop() é iniciado. O programa começa logo após a abertura da chave ({ }, e o processador vai executando as linhas de código até chegar na

chave de fechamento (}). Uma vez chegado ao fim, ele pula de volta para a primeira linha do loop e começa tudo de novo.

Variáveis e tipos de dados

Variáveis guardam valores (dados), podem ser números, textos e até tipos lógicos booleanos.

Existem variáveis globais → criadas fora de qualquer escopo, ou seja, fora de funções ou laços; E as locais → criadas dentro de um escopo, um exemplo claro é a variável criada dentro do laço *for*, que você verá mais a frente.

Para declarar uma variável usamos a seguinte sintaxe:

tipo nome = inicialização;

Uma variável inicializada é aquela que passamos um valor via código logo no início.

Números inteiros

Tipo	Tamanho	Intervalo de Valores
int	2 bytes	-32.768 a 32.767
unsigned int	2 bytes	0 a 65.535
long	4 bytes	-2.147.483.648 a 2.147.483.647
unsigned long	4 bytes	0 a 4.294.967.295
byte	1 byte	0 a 255

Números com ponto flutuante - decimais

Tipo	Tamanho	Precisão
float	4 bytes	6-7 dígitos
double	4 bytes	Mesmo que <i>float</i> no Arduino

Tipos lógicos (Booleanos)

Tipo	Tamanho	Valores Possíveis
------	---------	-------------------

bool	1 byte	true (1) ou false (0)
------	--------	-----------------------

Strings e Arrays

Tipo	Descrição
char[]	Vetor de caracteres (string de C)
String	Classe de manipulação de strings

Observe que char sem colchetes ([]) não é um array/vetor, então ele pode armazenar apenas um caractere.

```
//Declaração e inicialização de variáveis

int numeroInteiro = 0;

float numeroDecimal = 0.0;

char letra = "A"; //Armazena apenas um caractere

char arrayCaractere[] = "Olá, Mundo!"; /*Armazena texto pois
é um array de caracteres.*/

String Texto = "Olá, Mundo!";

bool verdadeiroFalso = true;
```

ATENÇÃO!!! A variável 'Texto' desobedece as convenções de nomenclatura, deveria ser 'texto'.

Observação: se adicionar um número nos colchetes, só poderá armazenar essa quantidade de caracteres, exemplo:

```
char letras[5] = "Teste";
```

Espaços também são caracteres, e Strings ocupam mais espaço, então pode ser preferível usar char quando se tem pouca memória RAM.

Arrays

Vamos nos aprofundar um pouco mais em Arrays. Array ou Vetor é uma variável que armazena um conjunto de dados do mesmo tipo.


```
void loop()
{
  int numArray[5] = {1, 2, 3, 4, 5};
}
```

Esses valores podem ser acessado usando seu índice:

Serial.println(numArray[0]);

O código acima acessa o valor que está na posição 1, ou seja o número 1 e imprime-o graças ao numArray[0].

O sistema de índice funciona da seguinte forma: posição que você quer do vetor - 1, pois o índice inicia em 0.

Arrays Bidimensionais

Arrays bidimensionais são matrizes, como mostra o exemplo abaixo:

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int l=2, c=2;
  int matriz[l][c] = {
    {1, 2},
    {3, 4}
  };
  for(int i = 0; i<l; i++){
    Serial.print("\n");
    for(int j = 0; j<c; j++){
      Serial.print(" ");
      Serial.print(matriz[i][j]);
    }
  }
  delay(5000);
}
```

Vamos explicar o código. Primeiramente criamos uma array normal, porém adicionamos dois blocos de colchetes ([]), o primeiro informa a quantidade de linhas e o segundo a quantidade de colunas. As chaves ({ }) definem os valores de cada posição. Podemos deixar sem as chaves para que seja uma matriz vazia e depois adicionamos os novos valores. Para imprimir os valores que estão em cada

linha de cada coluna usamos o **For Aninhado**, que você irá aprender mais a frente no material.

Operadores Básicos

Operadores são: Adição (+), Subtração (-), Divisão (/), multiplicação (*) e resto de (%)

Adição (+)

$$2 + 1 = 3$$

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int num1=2, num2=1;

  Serial.println(num1 + num2); //Imprime 3 no monitor serial
}
```

Subtração (-)

$$1 - 2 = -1$$

```
void loop()
{
  int num1=2, num2=1, resultSubtracao;
  //A variável que armazena o resultado não foi inicializada

  resultSubtracao = num2 - num1; /*Este exemplo foi feito com
  uso de uma segunda variável apenas para mostrar a possibilidade
  */
  Serial.println(resultSubtracao); //Imprime -1 no monitor serial
}
```

Divisão (/)

$$2 / 2 = 1$$

```
void loop()
{
  int num1=2, num2=0;

  Serial.println(num1/num1); //Imprime 1 no monitor serial
}
```

ATENÇÃO!!! num2 deveria receber o valor '2', não '0'.

$1 / 2 = 0.5$

```
void loop()
{
  float num1=1, num2=2;

  Serial.println(num1/num2); //Imprime 0.50 no monitor serial
}
```

Observe que o tipo foi mudado para float, isso foi necessário porque toda conta que retorna um número decimal tem que ter os números da operação configurados como decimais, caso contrário irá imprimir 0.00 ou algum outro número.

Caso haja uma divisão por zero é retornado INF que significa Infinito;

Caso haja a divisão de 0 por algum número o número impresso será 0;

Caso haja a divisão de 0 por 0 o resultado é NAN que significa *Not a Number* (Não é um Número).

Multiplicação (*)

$3 * 5 = 15$

```
void loop()
{
  float num1=3, num2=5;

  Serial.println(num1*num2); //Imprime 15 no monitor serial
}
```

Observação:

É mais aconselhável sempre usar float para operações, pois usando int em uma divisão por zero o resultado é -1, o que é um desvio no que seria “normal”, claro que você deve verificar o que está sendo usado nas operações usando condicionais, que é o próximo assunto da apostila.

Controle de Fluxo

É o conjunto de estruturas e comandos que determinam a ordem e as condições em que um programa executa suas instruções, ou seja, ele define como e quando certas partes do código serão executadas, com base em decisões, repetições e eventos.

Operadores lógicos

Operador	Nome	Exemplo	Explicação
&&	E(AND)	A && B	Só executa se A e B forem verdadeiros
	OU(OR)	A B	Executa se A ou B for verdadeiro
!	NÃO(NOT)	!A	Inverte o valor lógico

Operadores relacionais

Operador	Nome	Exemplo	Explicação
>	Maior que	A > B	Verdadeiro se A for maior que B
<	Menor que	A < B	Verdadeiro se A for menor que B
>=	Maior ou Igual	A >= B	Verdadeiro se A for maior ou igual a B
<=	Menor ou Igual	A <= B	Verdadeiro se A for menor ou igual a B
==	Igual a	A == B	Verdadeiro se A for igual a B
!=	Diferente de	A != B	Verdadeiro se A for diferente de B

Observe que na comparação de igualdade deve usar (==) e não (=).

Igual igual (==) é comparação; Igual (=) é atribuição, usado para dar valor à variável.

Estruturas condicionais

Comandos que permitem que um programa tome decisões com base em condições.

If e else

Estrutura básica:

```
void loop()
{
  |   if(condição){
      |       //seu código
    }else if(condição){
      |       //seu código
    }else{
      |       //seu código
    }
}
```

Se a condição do *if* for verdadeira, apenas o código dessa condição será executado, se a condição do *if* não for verdadeira, mas a do *else if* sim, apenas o código dessa condição será executado. Porém, se nem uma condição for atendida, o código que está dentro do *else* será executado.

Exemplo prático:

```
void loop()
{
  int a = 0, b = 1;

  if(b < a){
    Serial.println("A é maior");
  }else if(b == a){
    Serial.println("São iguais");
  }else{
    Serial.println("B é maior");
  }
}
```

Será impresso 'B é maior'.

Switch

Estrutura básica com valor inteiro:

```
void loop()
{
    int op = 1;
    switch(op) {
        case 1:
            //seu código
            break;
        case 2:
            //seu código
            break;
        default:
            //seu código
    }
}
```

Estrutura básica com caractere:

```
void setup()
{
    Serial.begin(9600);
}

void loop()
{
    char op = 'a';
    switch(op) {
        case 'a':
            //Seu código aqui
            break;
        case 'b':
            //Seu código aqui
            break;
    }
}
```

É uma estrutura de casos, ele depende de uma variável, no caso 'op', caso essa variável esteja armazenando o valor 1, o caso 1 será executado. Caso a variável não esteja armazenando nenhum dos valores dos casos, então o *default* será executado. É importante adicionar o comando *break*, ele para a execução para que não vá para os casos abaixo, por exemplo do caso 1 para o caso 2 e do caso 2 para o default (case 1 > case 2 > default).

O *default* não possui break porque é o último dos casos.

Exemplo prático com valor inteiro:

```
void loop()
{
    int op = 1;
    switch(op) {
        case 1:
            //seu código
            Serial.println("1");
            break;
        case 2:
            //seu código
            Serial.println("2");
            break;
        default:
            //seu código
            Serial.println("Nada");
    }
}
```

Será impresso no monitor serial o número 1.

Exemplo prático com caractere:

```
void setup()
{
    Serial.begin(9600);
}

void loop()
{
    char op = 'a';
    switch(op) {
        case 'a':
            Serial.write("Roda a");
            break;
        case 'b':
            Serial.write("Roda b");
            break;
    }
}
```

Será impresso no monitor serial a frase Roda a

Laços de Repetição

São estruturas que permitem executar um bloco de código várias vezes, sem precisar escrever as mesmas instruções repetidamente.

For

Estrutura básica:

```
void loop()  
{  
  for(inicialização; condição; atualização){  
    //seu código  
  }  
}
```

Inicialização → Declara e inicializa uma variável

Condição → Condição para execução com base na variável recém criada

Atualização → Atualiza a variável recém criada

Exemplo prático:

```
void loop()  
{  
  for(int i=0; i<5; i++){  
    Serial.println(i);  
  }  
}
```

Resultado impresso no monitor serial → 0 1 2 3 4

Este código será executado 5 vezes, pois 'i' foi inicializado como 0, se tivesse sido inicializado como 1 ele executaria 4 vezes. Quando 'i' for igual a 5 ele não executará, pois a condição diz para executar enquanto for menor que 5.

For Aninhado

O For Aninhado nada mais é do que um laço For dentro de outro. Exemplificando:


```

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  for(int i = 0; i<2; i++){
    for(int j = 0; j<2; j++){
      Serial.println("Oi");
    }
  }
  delay(5000);
}

```

O resultado será a impressão de “oi” 4 vezes, até que o código seja executado novamente por estar dentro de loop. Basicamente ele executa o primeiro for, vai para o segundo for e executa **duas** vezes, volta para o primeiro for, vai para o segundo for e executa **duas** vezes, pois quando ele vai para o primeiro e depois e entra no segundo, o valor da variável ‘j’ é definida como 0.

Isso é útil para trabalhar com arrays bidimensionais ou matrizes.

Por falar em **enquanto**, vamos para o próximo **Laço de Repetição**.

Observação:

$i++ \rightarrow i += 1$ → soma 1 e altera a variável (pós-incremento). Imaginando que $i=0$, o valor impresso será ‘0’ e só depois irá adicionar 1 à variável.

$++i$ → soma 1 instantaneamente e altera a variável. Imaginando que $i=0$, o valor impresso será ‘1’.

$i+1$ → soma 1 mas não altera a variável.

$i-- \rightarrow i -= 1$ → subtrai 1 e altera a variável (pós-decremento). Imaginando que $i=0$, o valor impresso será ‘0’ e só depois irá subtrair 1 da variável.

$--i$ → subtrai 1 instantaneamente e altera a variável. Imaginando que $i=0$, o valor impresso será ‘-1’.

$i-1$ → subtrai 1 mas não altera a variável.

$i *= 2$ → multiplica i por 2 e salva o resultado em i.

$i /= 2$ → divide i por 2 e salva o resultado em i.

While

Só executa quando a condição é verdadeira e continua executando enquanto ela for verdadeira.

Estrutura básica:

```
void loop()
{
  while(condição) {
    //seu código
  }
}
```

O código que está entre as chaves ({ }) do *while* será executado enquanto a condição for real.

Exemplo prático:

```
void loop()
{
  int num = 2;
  while(num > -2) {
    Serial.println(num);
    num--;
  }
}
```

Executa enquanto *num* for **maior que** -2. Este código será executado 4 vezes.
Resultado no monitor serial → 2 1 0 -1

Do While

Muito utilizado para fazer menus interativos.

Estrutura básica:

```
void loop()
{
    do{
        //seu código
    }while(condição);
}
```

Executa o que estiver entre as chaves ({ }) no **do**, depois verifica a condição no **while**, se for verdadeira executa novamente, senão ele para.

Exemplo prático:

```
void loop()
{
    int a = 0;
    do{
        Serial.println(a);
    }while(a != 0);
}
```

Este código irá rodar apenas uma vez, pois o **do** será feito e o **while** irá receber um *false*. Monitor serial: 0.

Funções

Imagine ter que escrever o mesmo código toda vez que for usá-lo em lugares diferentes do código, as desvantagens já são perceptíveis, código extenso, execução mais demorada, ocupa mais armazenamento, e a manutenção está em vários pontos diferentes, então temos as funções. Blocos de código que podem ser chamados em lugares diferentes no programa, quantas vezes quiser. Não confunda, embora void signifique vazio, trata-se de não haver retorno. Uma função vazia (void) pode ter parâmetros. E uma função com tipo pode não ter parâmetros

Estrutura básica:

```
void loop()
{
    //chamada da função com parâmetro e com retorno
    Serial.println(multiplicação(parâmetros));
}

//criação da função com parâmetro
int multiplicacao(parâmetros) {
    seu código
    return algo;
}
```

O tipo *int* antes do nome da função significa que é uma função com retorno do tipo **inteiro** e o 'parâmetros' dentro dos parênteses '(') mostra que há a passagem de parâmetros. Uma função pode ou não ter parâmetros.

```
void loop()
{
    ola(); //chamada da função sem parâmetro e sem retorno
}

//criação da função sem parâmetro
void ola() {
    //seu código
}
```

O tipo *void* antes do nome da função demonstra que não há retorno de nenhum tipo. Os parênteses '(') vazios demonstram que não há passagem de parâmetros. As funções podem ou não ter parâmetros.

Exemplo prático:

```
void loop()
{
    int num1=4, num2=3;
    //chamada da função com parâmetro e com retorno
    int result = multiplicacao(num1, num2);
    delay(1000);
    ola(); //chamada da função sem parâmetro e sem retorno
    Serial.println(result);
}

//criação da função com parâmetro
int multiplicacao(int numerol, int numero2){
    return numerol*numero2;
}

//criação da função sem parâmetro
void ola(){
    Serial.println("Olá, Mundo! -->> Hello, World!");
}
```

Monitor serial: Olá, Mundo! - ->> Hello, World! 12

Observe que podemos chamar uma função que tem retorno dentro de uma variável ou direto no comando para imprimir no monitor serial. O *return* envia o valor para quem o chamou, neste caso a variável, mas poderia ser o comando de impressão. A função de tipo *void*, por não ter retorno, não pode ser chamada em uma variável ou comando de impressão, uma função desse tipo cumpre seu serviço e não retorna nenhum valor. (Não significa que uma função sem retorno não imprima dados no monitor serial ou realize cálculos).

Eletrônica Básica para Arduino

Tensão (Voltagem) - V

- É a diferença de potencial elétrico entre dois pontos de um circuito.
- Medida em Volts (V).
- No Arduino, geralmente trabalhamos com 5V ou 3,3V.

Corrente Elétrica - I

- É o fluxo de elétrons através de um condutor.
- Medida em Ampères (A).
- No Arduino, a corrente fornecida pelos pinos digitais é limitada.

Resistência Elétrica - R

- Dificulta a passagem da corrente em um circuito.
- Medida em Ohms (Ω).

Exemplo: Um LED precisa de um resistor para limitar a corrente e evitar que ele queime.

Potência Elétrica - P

- Mede a energia consumida ou dissipada.
- Unidade: Watts (W).

Exemplo: Um motor conectado ao Arduino precisa de uma fonte de energia adequada para fornecer potência suficiente.

Componentes Comuns no Arduino

- LEDs: Emitem luz, precisam de resistores.
- Resistores: Limitam corrente e protegem componentes.
- Botões: Permitem interação, geralmente usados com resistores pull-up/down.
- Sensores: Leem informações do ambiente (temperatura, luz, movimento).
- Motores e Atuadores: Precisam de transistores ou drivers para serem controlados pelo Arduino.

Pinos do Arduino e Sua Função

- 5V e 3,3V: Alimentação para sensores e módulos.
- GND (Ground): Referência de terra do circuito.
- Pinos Digitais (0-13): Entrada e saída de sinais digitais (0 ou 1).
- Pinos Analógicos (A0-A5): Leem variações de tensão (0V a 5V).
- PWM (~): Simula sinais analógicos em pinos digitais.

Prática

Potenciação

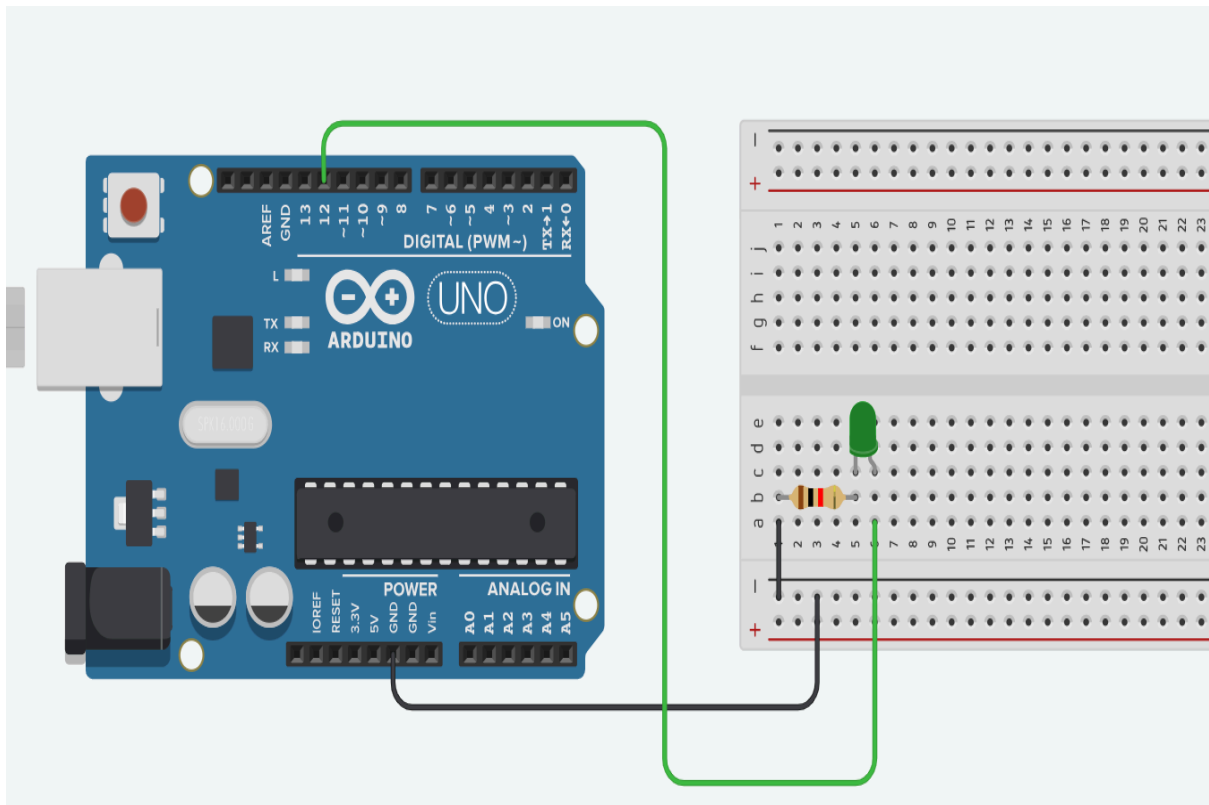
Faça um código que pergunte ao usuário o expoente que ele deseja elevar o número 2, guarde os resultados de cada iteração em um array, ao fim apresente os valores da array para o usuário.

```
void setup()
{
  Serial.begin(9600);
  Serial.println("Elevar a qual potencia?");
}

void loop()
{
  int entrada, a = 2;
  if(Serial.available()){
    entrada = Serial.parseInt(); //Converte String para Inteiro
    int resultado[entrada];
    Serial.print("Resultado: ");
    for(int i=0; i<entrada; i++){
      a*=2;
      resultado[i] = a;
      Serial.print(resultado[i]);
      Serial.print(" ");
    }
    Serial.println("\nElevar a qual potencia?");
  }
}
```

Observações importantes: o `parseInt` está sendo usado para converter a string (entrada do usuário) em número inteiro. Como em C++ não se pode criar um array de tamanho desconhecido como em Python e JavaScript, mudamos seu tamanho em tempo de execução, usando a entrada do usuário. O exemplo acima é um pouco redundante, haja vista que seria necessário apenas imprimir o valor que está em 'a' a cada iteração, mas a ideia era mostrar um uso interessante com arrays. Possibilidade de melhora: perguntar ao usuário qual o número que ele deseja elevar.

Blink LED



Circuito elétrico. O Pino 12 está ligado ao Anodo e o GND está conectado ao Cátodo. O resistor está entre o GND e o Cátodo.

```
// Define o pino onde o LED está conectado
const int LED_PIN = 12;

void setup() {
  // Configura o pino do LED como saída
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  // Liga o LED
  digitalWrite(LED_PIN, HIGH);
  // Aguarda 1 segundo
  delay(1000);

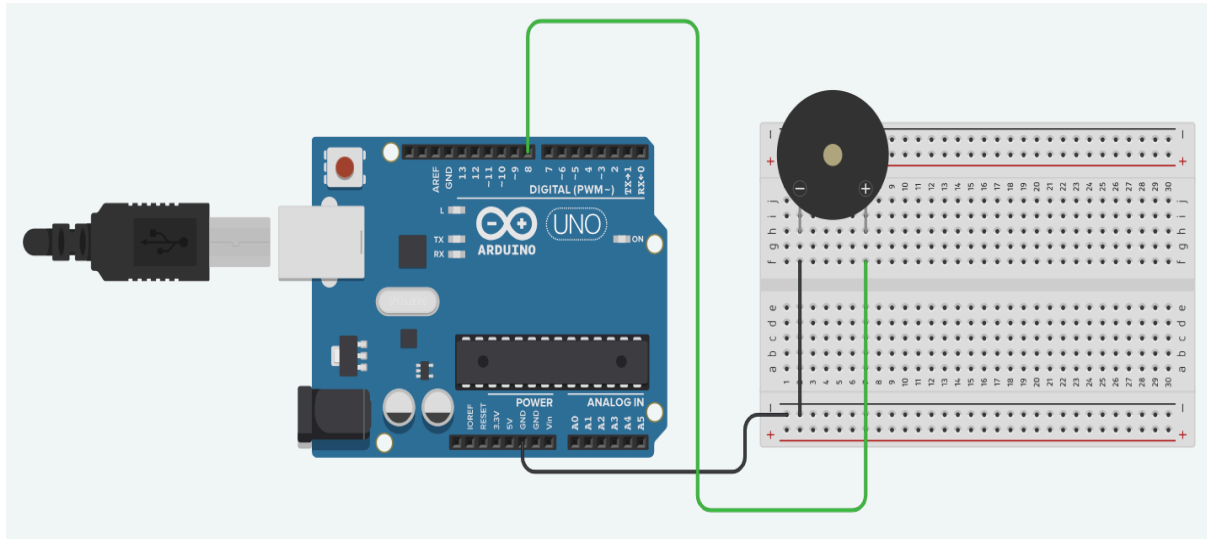
  // Desliga o LED
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```

Código.

const → Constante para o pino do LED

digitalWrite → Comando que envia sinal aos pinos digitais
delay → Tempo de espera, configurado em milissegundos
HIGH → pode ser trocado por 1, significa alta tensão
LOW → pode ser trocado por 0, significa baixa tensão

Buzzer



Circuito: positivo no pino 8, negativo no GND

```

// Define o pino onde o BUZZER está conectado
const int BUZZER_PIN = 8;
//Cria uma constante de tempo de 1 segundo
const int tempo = 1000;
const float c2 = 130.812775; // frequência da nota em Hz
const float e2 = 164.813782; // frequência da nota em Hz

void setup() {
    // Configura o pino do BUZZER como saída
    pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
    // Faz o BUZZER emitir som da nota C2
    tone(BUZZER_PIN, c2, tempo/4); // toca por 1/4 segundo
    // Aguarda 500 milissegundos
    delay(tempo/2);
    //Para o som do buzzer
    noTone(BUZZER_PIN);
    delay(tempo/2);
    tone(BUZZER_PIN, e2); //toca até que noTone() seja chamado
    delay(tempo);
    noTone(BUZZER_PIN);
    delay(tempo/2);
}

```

‘Código.

Função *tone()*, faz o Buzzer executar uma nota, sintaxe:

tone(Pino, Frequência, Tempo);

tone(Pino, frequência);

Aqui temos o chamado Overload (sobrecarga), é quando criamos funções com mesmo nome mas com diferentes parâmetros.

Frequência: dada em Hz

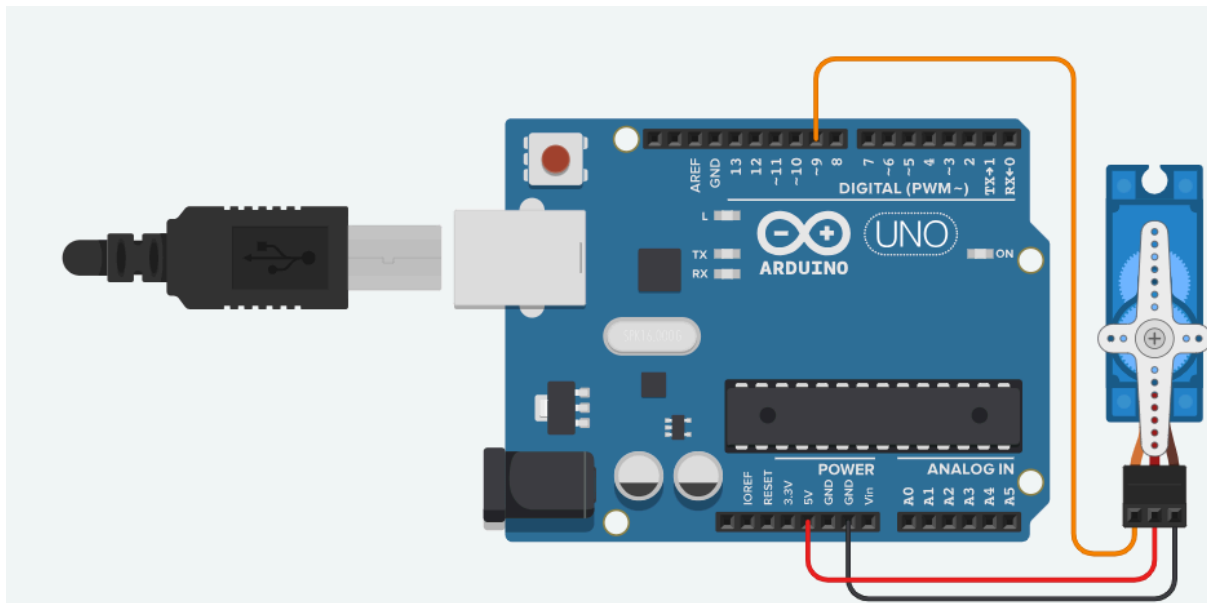
Tempo: tempo que a nota ficará tocando

Função *noTone()*, faz o Buzzer parar de tocar, sintaxe:

noTone(Pino);

Servo Motor

Possui limite de 0° até 180°.



Circuito: Sinal → jumper laranja conectado à porta 9

```
//Adciona a biblioteca do Servo Motor
#include<Servo.h>

Servo myservo; //Cria o objeto

void setup() {
  // Configura o pino do SERVO no objeto
  myservo.attach(9);
  Serial.begin(9600);
}

void loop(){
  origem(); // manda o servo motor de volta para a origem
  //Envia o servo para sua posição máxima (180)
  for(int pos=myservo.read(); pos<=180; pos++){
    myservo.write(pos);
    Serial.println(pos);
    delay(200);
  }
}

//Criação da função que manda a posição do servo para a origem
void origem(){
  myservo.write(0);
  delay(200);
}
```

`#include` → diz para o código usar determinada biblioteca.

`Servo.h` → Biblioteca para controlar o **Servo Motor**.

`myservo` → é o nome dado ao objeto que criei, poderia ser qualquer nome.

`attach()` → observe que invoco o nome do objeto e então chamo a função. Esta é uma função da **classe Servo**, ele adiciona ao objeto que criei o pino no qual o Servo Motor está conectado. Sintaxe:

`nomeObjeto.attach(Pino);`

`read()` → função que lê em que ângulo o Servo Motor está. Sintaxe:

`nomeObjeto.read();`

`write()` → função que informa para que posição o Servo Motor deve ir. Sintaxe:

`nomeObjeto.write(Ângulo);`

Por que criar objetos? Pois é uma biblioteca criada para trabalhar com objetos. Além de que é isso que permite o trabalho com vários Servos de forma independente.

Observe que é a mesma coisa do `Serial.begin(9600)`, é a função de uma classe sendo chamada, mas no caso da Serial ela já é uma classe pré-definida, ou seja, não há necessidade (nem possibilidade) de criar um objeto.

OBSERVAÇÃO: O `Serial.println(pos)` é apenas um debug, ou seja, serve apenas para verificar se as posições estão sendo enviadas corretamente.